Nirikshan: Process Mining Software Repositories to Identify Inefficiencies, Imperfections, and Enhance Existing Process Capabilities

Monika Gupta
Indraprastha Institute of Information Technology, Delhi (IIITD)
New Delhi, India
monikag@iiitd.ac.in
http://www.iiitd.edu.in/~monikag/

ABSTRACT

Process mining is to extract knowledge about business processes from data stored implicitly in ad-hoc way or explicitly by information systems. The aim is to discover runtime process, analyze performance and perform conformance verification, using process mining tools like ProM and Disco, for single software repository and processes spanning across multiple repositories. Application of process mining to software repositories has recently gained interest due to availability of vast data generated during software development and maintenance. Process data are embodied in repositories which can be used for analysis to improve the efficiency and capability of process, however, involves a lot of challenges which have not been addressed so far. Project team defines workflow, design process and policies for tasks like issue tracking (defect or feature enhancement), peer code review (review the submitted patch to avoid defects before they are injected) etc. to streamline and structure the activities. The reality may not be the same as defined because of imperfections so the extent of non-conformance needs to be measured. We propose a research framework 'Nirikshan' to process mine the data of software repositories from multiple perspectives like process, organizational, data and time. We apply process mining on software repositories to derive runtime process map, identify and remove inefficiencies and imperfections, extend the capabilities of existing software engineering tools to make them more process aware, and understand interaction pattern between various contributors to improve the efficiency of project.

Categories and Subject Descriptors

D.2.2 [**Software Engineering**]: Design Tools and Techniques—*petri nets, state diagrams*; D.2.8 [**Software Engineering**]: Metrics—*process metrics, performance measures*; D.2.9 [**Software Engineering**]: Management—*life cycle, software process models*

ICSE Companion'14, May 31 – June 7, 2014, Hyderabad, India
Copyright 2014 ACM 978-1-4503-2768-8/14/05...$15.00
http://dx.doi.org/10.1145/2591062.2591080

General Terms

Algorithms, Design, Management, Performance, Measurement, Verification

Keywords

Mining Software Repositories, Process Mining, Business Process Intelligence, Open-Source Software, Empirical Software Engineering and Measurements

1. RESEARCH MOTIVATION AND AIM

Software developers and maintenance professionals use several Process Aware Information Systems (PAIS) such as Issue Tracking System (ITS), Peer Code Review (PCR) system and Version Control System (VCS) during software development and maintenance. These systems store process data, using which, the event log can be generated for process mining. We consider process as the way an organization arranges the order of activities and the group of people to whom specific activities to be allocated for successful accomplishment [15]. Process mining is concerned with the extraction of knowledge about business processes from system logs [12]. Process mining should be applied to software repositories, however, involves a lot of challenges. Business Process Intelligence (BPI) refers to the application of various measurement and analysis techniques in the area of business process management on business processes. We advocate the application of process mining and BPI that can help software industries to detect trends within their own operations on the basis of the vast amounts of data processed and stored by software development systems. This research work is motivated by the need to process mine software repositories with the objective to identify inefficiencies and imperfections from runtime (reality) process map discovered using event log. We perform process mining from organizational perspective to identify organizational roles, interaction pattern, and work distribution among performers (resource) and activities executed by a particular resource. Also improve the process awareness of existing software engineering tools, deployed for various software development activities (like ITS, VCS etc.) thus, facilitating direct process mining. These applications of process mining can ensure better process maintainability, capability, reliability, productivity, efficiency and stability [4] [5]. Specific research aims of the work presented in this paper are following:

1. Survey practitioners to understand the existing process practices, identify research gaps and validate initial hypothesis.
2. To address identified challenges by novel applications of process mining. Process mining will be applied to software repositories for process discovery, identification of inefficiencies and imperfections. Framework will be designed to help solve well known maintenance professional's problems like workload management. Existing software tools will be extended for better process awareness and process mining compatibility to facilitate business analysts.
3. To implement the solution approach on real world data and get feedback from practitioners to inform research.

2. RELATED WORK

In this section, we present brief literature review of the work closely related to the proposed research and identify the research gaps. Firstly, we study the literature where process mining is applied on software repositories. The most closely related research to the process mining of single repository is by Sunindyo *et al.* [10] in which they proposed a framework for effective engineering process observation in FLOSS (Free/Libre and Open Source Software) projects. They used hypothesis testing approach to verify design model with runtime event log from bug history data. They obtained process map for RHEL bug history data using heuristic mining algorithm implemented in process mining tool, ProM. The conformance verification is not in-depth and done for few dimensions by proving/disproving the hypothesis. Poncin *et al.* [7] combined the information from different repositories by matching related events. This is achieved by a prototype FRASR (FRamework for Analyzing Software Repositories). The resulting log is analyzed using ProM dotted chart visualization and fuzzy miner plug-in for role classification and fuzzy graph generation (actual view of bug lifecycle) respectively. An interactive approach to visualize effort estimation and process lifecycle patterns in ITS (for an industrial project) to detect outliers, flaws and interesting properties is proposed [5]. Akman *et al.* [1] analyzed and compared the effectiveness of four process discovery algorithms (Markov model, Heuristic miner, Fuzzy miner, Genetic mining) for software organization. They performed a case study on configuration management tool of an industry project (for Turkish Armed Forces) and discussed the discrepancies between derived real time and defined design time process. Rubin *et al.* [9] performed Linear Temporal Logic (LTL) checking, social network discovery, performance analysis and petri net discovery for configuration management tool and version control system of AgroUML (an open source) project.

From literature, we observe that the data of software repositories is relatively unexplored for process mining from multiple perspectives like process (control), organizational, time and data/case etc. [13]. Some research from organizational and process perspective has been done. Few challenges related to data incompatibility with process mining tools identified in literature [7], however, not fully addressed.

3. RESEARCH METHODOLOGY AND PROPOSED CONTRIBUTIONS

We present the research framework *Nirikshan* (a Sanskrit word which means 'to investigate') in Figure 1 showing proposed approach and research contributions. We extract the process data from software repositories such as issue tracking system (like Bugzilla, JIRA), peer code review system (Rietveld, Gerrit), version control system (Mercurial, Git), project hosting platforms (sourceforge) and community based QnA websites (stackoverflow), used by developers during software development and maintenance activities. Data extraction is done using APIs (if available), XML-RPC, JSON-RPC, web scraping etc. We plan to survey practitioners and do field study to understand the process and policies in practice, to identify the problems encountered by them and validate our initial hypothesis. We believe that this will strengthen the work as practitioner's requirements are clearly understood. Figure 1 depicts four research directions that we plan to investigate: process mining software repositories, explore organizational perspective, extending the capabilities of existing SE tools, and application to solve well known software maintenance professional's problems.

1. **Process Mining Software Repositories**: Many Process Aware Information Systems are used to support business processes in software industry. For example, issue tracking system is a PAIS to support the business process of bug resolution. Process instances are identified by unique caseID and a list of activities pertaining to the same instance. Every process instance involves many activities performed by various actors (performer as resource "who"), acting for a specific role at specific timestamp (ts). For instance, a bug (caseID) lifecycle starting from reporting to resolution involves one activity of bug triaging by triager (role) performed by a member x (actor: who) at specific timestamp (ts). As shown in Figure 1, there are two types of scenario:

   (a) Process mining data from single repository if complete process lifecycle is limited to one information system. For example, an issue is reported in ITS; it undergoes transitions and state changes during it's lifecycle till resolution, however, limited to one system.
   (b) Process data extracted from multiple repositories as some processes span across multiple systems with few activities performed in one system and remaining in other to complete the task. One such instance is when a bug is reported in ITS and to fix the bug, a patch is submitted to peer code review system. The reviewed patch is committed to source code repository (VCS). So the process spans across three systems.

   For both the above situations, to perform process mining, we extract process data which includes case ID, activity, timestamp, actor and role. Extracted data is transformed to event log and then process mining is applied using tools and framework like ProM[1] (open source) and Disco[2] (commercial). The transformation is very crucial for process mining due to incompatibility of extracted data with existing process mining tools. We address challenges like missing value by imputation and resolve time conflicts. The resultant event log is applied for process mining as follows:

[1]http://www.promtools.org/prom6/
[2]http://fluxicon.com/disco/

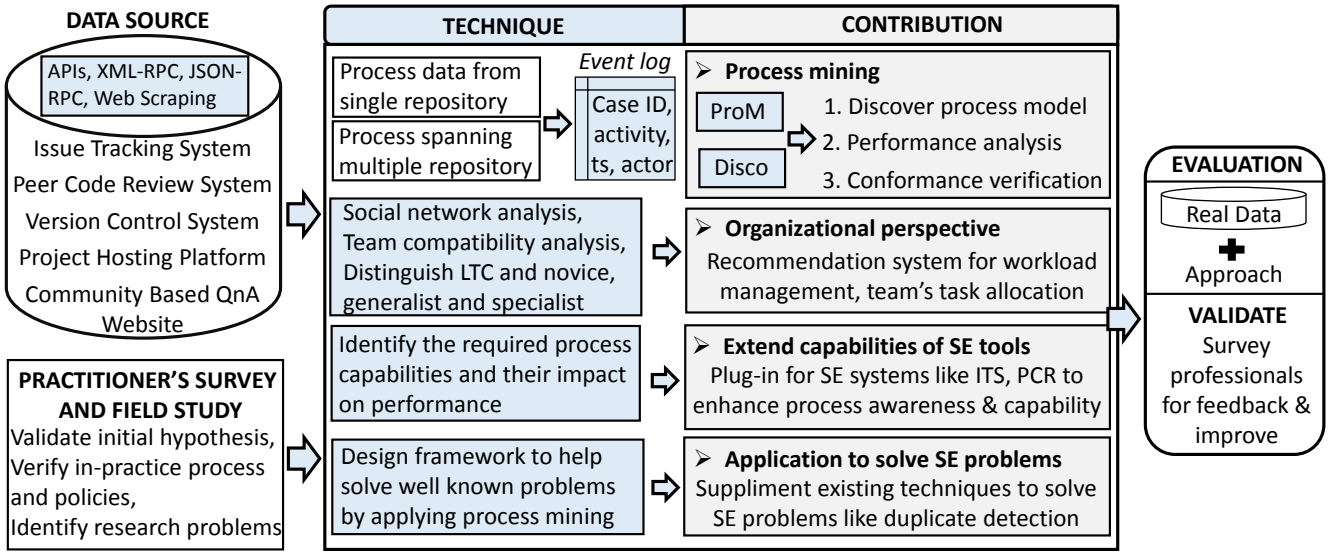

Figure 1: Nirikshan: Research framework for the research contributions: 1. Process mining single repository, 2. Process mining multiple repositories, 3. Organizational perspective, 4. Extension of SE tool capabilities and 5. Application to solve SE maintenance professional's problems, followed by Evaluation.

(a) *Discover runtime process model*: It shows as-is process (reality) obtained from the runtime event log. Statistical analysis of the number of unique traces, and activity and transition frequency distribution is done. There are many process map generation algorithms like heuristic miner [15], α - algorithm [11], genetic algorithm [2] etc. implemented as plug-in in ProM which can be selected according to the complexity of process. It is the first step for effective business analysis.

(b) *Performance analysis*: We analyze process to find inefficiencies and imperfections including identification of bottlenecks, redundant activities, loops showing undesired repetition of activities, unsatisfactory performance by specific actors etc. This helps in streamlining the process and hence reduce the flow time and shorten the response time.

(c) *Conformance verification*: We measure how well discovered process map complies with the defined design process using algorithm and metrics. Also we identify the point of inconsistency to improve conformance. We believe that it can be useful for identification and removal of obsolete parts which demands extra maintenance efforts [8].

2. **Analyze organizational perspective**: We understand the dynamics of an organization (who are performers of the activities and how they are related), which is also a dimension of process mining, by performing social network analysis. We plan to structure an organization by classifying people as specialists or generalists and long term contributors (LTC) [16] or novice and explore their association with the process performance. Also, we will study the compatibility between various team members as better cooperation improves the throughput [6]. This can help manage changing workloads and efficient task allocation.

3. **Extend capabilities of existing software engineering tools**: The existing software engineering systems like ITS and PCR system are widely used during software maintenance. We will enhance capabilities of these systems by designing plug-in to make them more process aware with increased throughput. We will identify the features (attributes) having impact on overall performance and incorporate them in plug-in. Like, existing systems do not store history in format compatible with process mining tools so a plug-in can be designed for logging event history and measure performance using defined metrics, thus, ensure quality.

4. **Apply process mining for solving well known maintenance professional's problems**: There are well known challenges encountered by maintenance professionals like bug localization, duplicate detection etc. We aim to provide a fresh perspective demonstrating the usefulness of process mining to solve these problems. The objective is to suppliment the existing Natural Language Processing and Information Retrieval based techniques for improvement.

Tools, algorithm, metrics, plug-in, recommendation system, prediction model and framework created as result of proposed approach. They are evaluated during the last phase both empirically and qualitatively.

Evaluation: We implement the proposed approach on real data from software repositories of large open source (FLOSS) and commercial projects. Since the work is application based and industry oriented, we aim to do validation through surveys and interaction with practitioners for their feedback followed by improvement.

As shown in Figure 1, the proposed research work makes following novel and unique research contributions:

1. To the best of our knowledge, the research work proposed in 'Nirikshan' is the first in-depth attempt on

process mining event logs mined from software repositories for discovering runtime process maps, inefficiencies and inconsistencies.

2. Analysis from organizational perspective to understand team compatibility, perform delta analysis between the process adopted by long term contributors (LTC) and novice (specifically for open source), identify specialists and generalists. Thereafter, design a recommendation system for better workload management.
3. Plug-in for existing software engineering tools like ITS to extend the process capabilities thus, enabling timely preventive actions.
4. Demonstrate the usefulness of process mining in solving problems like duplicate bug detection faced by software maintenance professionals.

4. PRELIMINARY RESULTS

Preliminary work demonstrates the feasibility of process mining software repositories and the potential dimensions for research work. A brief account of the work submitted and the work in progress is presented in this section.

We applied business process mining tools and techniques to analyze the event log data (bug report history) generated by an issue tracking system with the objective of discovering runtime process maps, inefficiencies and inconsistencies [3]. We conduct a case-study on data extracted from Bugzilla ITS of the popular open-source Firefox browser and Core project. We perform a series of process mining experiments to study self-loops, back-and-forth, issue reopen, unique traces, event frequency, activity frequency, bottlenecks and present an algorithm to compute the degree of conformance between the design time and the runtime process. The runtime process map derived from event logs of 12234 Firefox and 24253 Core issue reports, reveal state transitions and event traces which are common and also the traces which are inconsistent with the design time process model. Empirical analysis reveals the distribution of 15 different activities and we infer that a statistically significant percentage of bug reports undergo component, developer and QA manager reassignment. We infer that majority of the bugs have $3-4$ events in lifecycle. Number of unique traces from start to finish is large (1164 and 622 variants for Core and Firefox respectively) and the top 2 unique traces constitute more than 25% and 33% of the Firefox and Core bug reports respectively. We find that several bug reports with status *wontfix* and *worksforme* are getting reopened. We observe self-loop and back-and-forth transitions (undesirable and inefficient) for some states. We identify bottlenecks to inform process owner for improvement. We discover the value of fitness metric as 0.86 and 0.91 for Core and Firefox respectively using the proposed conformance checking algorithm.

We are working on a project involving multiple software repositories. Some issues reported in ITS require patch for resolution which has to be peer reviewed before committing to source code repository to avoid defects before they are injected into the software. The process followed from the inception (issue reported in ITS) till resolution is studied for Google Chromium project which involves process mining data from three repositories: Google Chromium ITS, Rietveld PCR system and VCS. We identify bottlenecks, define and detect basic and composite anti-patterns. In addition to control flow analysis, we mine event log to perform organizational analysis and discover metrics such as handover of work, subcontracting, joint cases and joint activities [14].

5. ACKNOWLEDGEMENT

The presented research work is supported by Prime Minister's Fellowship awarded to the author. I thank my PhD advisor, Dr. Ashish Sureka and industry mentor, Dr. Srinivas Padmanabhuni for valuable inputs. I acknowledge SERB, CII and Infosys Limited for their support.

6. REFERENCES

[1] B. Akman and O. Demirors. Applicability of process discovery algorithms for software organizations. In *Euromicro Conference on SEAA '09.*, pages 195–202, 2009.

[2] Carmen Bratosin, Natalia Sidorova, and Wil van der Aalst. Distributed genetic process mining. In *IEEE CEC*, pages 1–8. IEEE, 2010.

[3] M. Gupta and A. Sureka. Nirikshan: Mining bug report history for discovering process maps, inefficiencies and inconsistencies. In *Seventh ISEC*, 2014.

[4] Ekkart Kindler, Vladimir Rubin, and Wilhelm Schafer. Activity mining for discovering software process models. In *Software Engineering*, volume 79 of *LNI*, pages 175–180. GI, 2006.

[5] Patrick Knab, Martin Pinzger, and Harald C. Gall. Visual patterns in issue tracking data. In *Proceedings of New modeling concepts for today's software processes: software process*, ICSP'10, pages 222–233. Springer-Verlag, 2010.

[6] Akhil Kumar, RM Dijkman, and Minseok Song. Optimal resource assignment in workflows for maximizing cooperation. *Business Process Management, Lecture Notes in Computer Science*, 8094:235–250, 2013.

[7] W. Poncin, A. Serebrenik, and M. van den Brand. Process mining software repositories. In *CSMR*, pages 5–14, 2011.

[8] Anne Rozinat and Wil MP van der Aalst. Conformance checking of processes based on monitoring real behavior. *Information Systems*, 33(1):64–95, 2008.

[9] Vladimir Rubin, Christian W. Günther, Wil M. P. Van Der Aalst, Ekkart Kindler, Boudewijn F. Van Dongen, and Wilhelm Schäfer. Process mining framework for software processes. In *ICSP*, pages 169–181. Springer-Verlag, 2007.

[10] Wikan Sunindyo, Thomas Moser, Dietmar Winkler, and Deepak Dhungana. Improving open source software process quality based on defect data mining. In *Software Quality. Process Automation in Software Development*, pages 84–102. Springer, 2012.

[11] Wil Van der Aalst, Ton Weijters, and Laura Maruster. Workflow mining: Discovering process models from event logs. *IEEE Transactions on Knowledge and Data Engineering*, 16(9):1128–1142, 2004.

[12] Wil MP van der Aalst. Process mining - discovery, conformance and enhancement of business processes. *Springer*, 2011.

[13] Wil MP van der Aalst. A decade of business process management conferences: personal reflections on a developing discipline. In *Business Process Management*, pages 1–16. Springer, 2012.

[14] Wil MP Van Der Aalst, Hajo A Reijers, and Minseok Song. Discovering social networks from event logs. *CSCW*, 14(6):549–593, 2005.

[15] AJMM Weijters, Wil MP van der Aalst, and AK Alves De Medeiros. Process mining with the heuristics miner-algorithm. *Technische Universiteit Eindhoven, Technical Report WP*, 166, 2006.

[16] Minghui Zhou and Audris Mockus. What make long term contributors: Willingness and opportunity in oss community. In *Proceedings of the 2012 International Conference on Software Engineering*, pages 518–528. IEEE Press, 2012.